

IUT de Niort — Université de Poitiers

BUT Sciences des Données — Année universitaire 2025–2026

RAPPORT DE STAGE

Conception et développement d'un mini-ERP de production sous WordPress

De l'Excel partagé à l'outil de gestion industrielle

⚙️ **Mission 1** — Plugin « Suivi Commandes » (suivi de production)

🚚 **Mission 2** — Outil de calcul et de conversion transport

Stagiaire	Rémi Misery
Tuteur entreprise	Fabien Benetreau
Tuteur pédagogique	Eric Payet
Période de stage	7 avril — 12 juin 2026
Entreprise	Tikimob

Remerciements

Je tiens à remercier l'ensemble de l'équipe Tikimob pour son accueil chaleureux et sa confiance tout au long de ce stage. J'adresse mes remerciements les plus sincères à **Fabien Benetreau**, mon tuteur en entreprise, pour sa disponibilité et ses conseils précieux, qui m'ont permis de monter rapidement en compétences sur les problématiques métier de la production industrielle.

Je remercie également **Monsieur Eric Payet**, mon tuteur pédagogique, pour son suivi attentif et ses retours constructifs, qui ont contribué à orienter mon travail tout au long du stage.

Mes remerciements vont aussi aux **opérateurs de l'atelier de production**, qui ont pris le temps de m'expliquer leurs contraintes quotidiennes. Leurs retours de terrain ont été essentiels pour concevoir des outils réellement adaptés à leurs besoins, plutôt que de simples exercices techniques déconnectés de la réalité de la production.

Enfin, je remercie l'ensemble du corps enseignant du département **Sciences des Données de l'IUT de Niort**, qui m'a transmis les bases méthodologiques nécessaires à la conduite de ce projet, tant sur le plan technique (développement web, bases de données, modélisation) que sur le plan analytique.

Résumé

Ce rapport présente le travail réalisé lors de mon stage au sein du groupe **Tikimob**, acteur français de la fabrication et de la vente en ligne de mobilier en kit sur mesure. Mon stage s'est articulé autour de deux missions complémentaires.

Mission 1 (principale) : la conception et le développement d'un outil de suivi de production sous forme de plugin WordPress, baptisé « Suivi Commandes ». Cet outil remplace un fichier Excel partagé entre les postes de l'atelier et centralise les commandes provenant de plus de treize boutiques WooCommerce dans un tableau de bord de production unifié. Développé en PHP (back-end) et Vue.js (front-end), il implémente un workflow de production complet, un système de rôles granulaires (RBAC), un mécanisme d'import multi-sites via l'API REST WooCommerce et un chemin retour assurant la synchronisation bidirectionnelle des données vers les sites d'origine. Le projet s'est enrichi de nombreux modules : tableau de bord analytique, archivage automatique, tickets SAV, gestion des paiements en plusieurs fois (acompte/solde) et une suite complète d'exports industriels.

Mission 2 (secondaire) : la reprise et la modernisation d'un outil Excel à macros VBA, « Outil de conversion transport », qui transforme un fichier brut issu du logiciel de fabrication en une série de fichiers exploitables par l'atelier et les transporteurs. Cette mission a consisté à comprendre, documenter puis ré-implémenter en PHP la logique métier (calcul de poids des panneaux, regroupement des pièces en colis, routage transporteur par département), afin de l'intégrer directement au plugin de la mission 1.

Au-delà du gain de productivité mesuré (de l'ordre de **80 minutes par jour et par opérateur**), ce stage m'a permis de mener un projet de bout en bout : analyse du besoin, modélisation, développement full-stack, débogage en production et dialogue continu avec les utilisateurs finaux.

Mots-clés : *ERP, WordPress, WooCommerce, Vue.js, API REST, gestion de production, plugin, workflow industriel, RBAC, ACF, synchronisation bidirectionnelle, VBA, EDI transport.*

This report presents the work carried out during my final-year internship at **Tikimob**, a French e-commerce group specialised in custom-made flat-pack furniture. The internship was built around two complementary missions.

Mission 1 (main) consisted in designing and developing a **production-tracking tool** as a WordPress plugin, named « Suivi Commandes ». It replaces a shared Excel file and centralises orders from more than thirteen WooCommerce stores into a single, unified production dashboard. Built with **PHP** (back-end) and **Vue.js** (front-end), the plugin implements a full production workflow modelled as a state machine, a granular **Role-Based Access Control** (RBAC) system, a multi-site remote import mechanism based on the WooCommerce REST API, and a **two-way synchronisation channel** (write-back) that keeps the source stores up to date. It was further enriched with an analytics dashboard, an automatic archiving system, after-sales (SAV) tickets, split-payment handling (deposit/balance) and a complete

suite of industrial exports (carrier EDI files, Sormec and Panotech machining files, delivery notes, labels).

Mission 2 (secondary) involved **porting a legacy VBA Excel tool** to PHP — panel-weight computation, parcel grouping (« collision » algorithm) and department-based carrier routing — and integrating it directly into the plugin so that operators no longer need to open Excel.

Beyond the measured productivity gain (around **80 minutes per operator per day**), this internship allowed me to lead a project end-to-end: requirements analysis, data modelling, full-stack development, production debugging and continuous dialogue with end users.

Keywords: *ERP, WordPress, WooCommerce, Vue.js, REST API, production management, plugin, industrial workflow, RBAC, two-way sync, VBA, EDI.*

Table des matières

Remerciements	i
Résumé / Abstract	ii
Introduction	2
1 Présentation de l'entreprise	3
1.1 Le groupe Tikimob	3
1.1.1 Quelques repères économiques	3
1.2 Les sites e-commerce du groupe	3
1.3 L'atelier de production	4
1.4 Un atelier fortement automatisé	5
1.5 Organisation informatique	6
2 Contexte et problématique	7
2.1 Un cahier des charges à approfondir	7
2.2 La situation de départ : le fichier Excel partagé	7
2.2.1 Structure de l'ancien fichier	7
2.2.2 Les limites identifiées	7
2.3 Estimation du temps perdu	8
2.4 Le cahier des charges consolidé	8
3 Architecture technique (Mission 1)	9
3.1 Vue d'ensemble	9
3.2 Stack technologique	9
3.3 Structure des fichiers du plugin	9
3.4 Modèle de données	10
3.4.1 Schéma de la base	10
3.4.2 Choix de modélisation	10
4 Développement front-end (Mission 1)	12
4.1 L'application Vue.js	12
4.2 Des vues adaptées à chaque poste	12
4.3 Fonctionnalités d'interface avancées	12
5 Développement back-end (Mission 1)	14
5.1 Une API REST sur mesure	14
5.2 Intégration avec WooCommerce	14
5.2.1 Les hooks natifs	14
5.2.2 Le décodage des dimensions	15
5.2.3 Le calcul de la DSU	15

5.3	La traduction des statuts (status-mapper)	15
5.4	Le plugin compagnon sc-expose-meta.php	15
5.5	Le système de rôles (RBAC)	16
6	Import multi-sites et chemin retour (Mission 1)	17
6.1	Le défi de l'agrégation multi-sites	17
6.2	Le mécanisme d'import en trois phases	17
6.3	Le chemin retour (synchronisation bidirectionnelle)	17
7	Workflow de production (Mission 1)	19
7.1	La machine à états	19
7.2	La gestion de la non-conformité	19
7.3	L'avancement automatique	19
8	Modules avancés (Mission 1)	21
8.1	Tableau de bord analytique	21
8.2	Archivage automatique	21
8.3	Actualisation quotidienne	21
8.4	Tickets SAV et retours atelier	21
8.5	Paie en plusieurs fois : acompte et solde	21
8.6	Notifications internes et routes de diagnostic	22
9	Mission 2 — Outil de calcul et de conversion transport	23
9.1	Contexte de la mission 2	23
9.2	Les paramètres de l'outil	23
9.3	La logique métier reconstituée	23
9.3.1	Le référentiel des poids de panneaux	23
9.3.2	Le routage transporteur par département	24
9.3.3	La conversion du fichier brut	24
9.3.4	Le regroupement des pièces (le « calcul collision »)	25
9.4	Les fichiers produits	26
9.5	Le portage en PHP et l'intégration au plugin	27
10	Difficultés rencontrées et solutions	28
10.1	Problèmes de données et d'intégration	28
10.2	Problèmes d'interface	28
10.3	Problèmes de déploiement	29
10.4	La difficulté propre à la mission 2	29
11	Bilan technique et perspectives	30
11.1	Bilan technique	30
11.1.1	Documentation et passation	30
11.2	Ce qu'il reste à faire	30
11.3	Compétences acquises	31

Conclusion	32
Bilan professionnel	33
Bilan personnel	34
A Glossaire technique	35
B Extraits de code significatifs	37

Table des figures

1.1	Chaîne de production de l'atelier Tikimob	4
3.1	Architecture globale du système Suivi Commandes	9
7.1	Machine à états du workflow de production (avec boucle qualité)	19
9.1	Colisage : du classeur Excel à la page intégrée	24

Liste des tableaux

1.1	Chiffre d'affaires du groupe Tikimob (estimations)	4
1.2	Sites e-commerce du groupe Tikimob	4
2.1	Comparatif des temps : ancien processus vs. plugin Suivi Commandes	8
3.1	Technologies utilisées	10
3.2	Principaux fichiers du plugin	11
3.3	Tables principales de la base de données	11
4.1	Vues par rôle utilisateur	13
5.1	Échantillon des routes de l'API REST (60 au total)	14
5.2	Aperçu des rôles RBAC (une vingtaine au total)	16
9.1	Principaux paramètres de l'onglet Accueil	25
9.2	Structure du référentiel matières (valeurs masquées)	25
9.3	Fichiers de sortie produits par l'outil	26

Introduction

Dans le cadre de la dernière année de mon BUT Sciences des Données à l'IUT de Niort, j'ai effectué un stage au sein du groupe **Tikimob**, acteur français spécialisé dans la fabrication et la vente en ligne de mobilier en kit sur mesure. Cette expérience, qui s'est déroulée du 7 avril au 12 juin 2026, marque l'aboutissement de ma formation et conditionne la validation de mon diplôme.

À la différence de nombreux stages où la mission consiste à exploiter une infrastructure déjà en place, j'ai eu ici la chance de **concevoir des outils de A à Z**, en partant d'un constat simple : la production de Tikimob, qui regroupe plus d'une dizaine de sites e-commerce convergeant vers un atelier unique, était pilotée à l'aide d'un **fichier Excel partagé**. Ce fichier, malgré sa simplicité, atteignait ses limites : impossibilité pour deux personnes de le modifier en même temps, saisie manuelle source d'erreurs, absence de traçabilité et aucun contrôle d'accès. À mesure que le groupe se développait sur de nouveaux marchés européens, ces limites devenaient un véritable frein opérationnel.

Mon stage s'est organisé en deux temps, que ce rapport suit fidèlement :

1. **Mission 1 — le suivi de production.** Concevoir, développer et déployer un mini-ERP de production prenant la forme d'un plugin WordPress, « Suivi Commandes ». Cet outil agrège automatiquement les commandes de tous les sites du groupe, propose à chaque opérateur une vue adaptée à son poste, modélise le workflow de fabrication sous forme de machine à états, garantit la traçabilité grâce à un système de rôles, et synchronise les données dans les deux sens entre l'atelier et les boutiques en ligne.
2. **Mission 2 — le calcul et la conversion transport.** Reprendre un outil Excel truffé de boutons et de macros VBA, en comprendre la logique métier, puis la ré-implémenter en PHP pour l'intégrer au plugin. Cet outil prépare automatiquement les fichiers techniques destinés aux machines d'usinage et aux transporteurs.

Ce rapport s'adresse à un lecteur qui n'est pas forcément développeur. Chaque terme technique (API, webhook, RBAC, ACF, machine à états...) est donc expliqué la première fois qu'il apparaît, et un glossaire complet figure en annexe [A](#).

Le rapport se structure comme suit. Le chapitre [1](#) présente l'entreprise et sa chaîne de production. Le chapitre [2](#) détaille la situation de départ et le cahier des charges. Les chapitres [3](#) à [8](#) décrivent la mission 1. Le chapitre [9](#) est entièrement consacré à la mission 2. Le chapitre [10](#) revient sur les difficultés rencontrées et le chapitre [11](#) dresse le bilan et les perspectives.

Présentation de l'entreprise

1.1 Le groupe Tikimob

Tikimob est un **fabricant français** spécialisé dans la conception et la vente en ligne de **mobilier en kit sur mesure** et d'aménagement d'intérieur. Fort de plus de **30 ans d'expérience** dans la fabrication de meubles, le groupe s'inscrit dans une démarche contemporaine où le design, la fonctionnalité et la qualité de fabrication occupent une place centrale. Fondé autour de la marque éponyme Tikimob, il a progressivement étendu son activité à travers plusieurs marques et sites e-commerce couvrant différents segments de marché et différentes zones géographiques européennes.

Chaque meuble est **conçu à la commande et fabriqué en France**, à partir de matériaux fiables (principalement des panneaux mélaminés) et assorti d'une garantie de dix ans. En amont, un **configurateur 3D** développé en interne laisse le client définir lui-même les dimensions, l'agencement intérieur, les coloris et les finitions de sa création, avec un prix mis à jour en temps réel ; en aval, un bureau d'études traduit ces choix en plans techniques fabricables. Cette logique de production à la commande limite le gaspillage et donne naissance à des meubles uniques, pensés pour s'intégrer dans des configurations parfois complexes (sous-pente, sous-escalier, meubles d'angle, séparateurs de pièce, etc.).

L'originalité de Tikimob réside dans son modèle industriel : toutes les commandes, quelle que soit leur provenance (site français, espagnol, allemand ou italien), convergent vers un **atelier de production unique** où les meubles sont fabriqués sur mesure, puis emballés en kit avant expédition. C'est précisément cette centralisation industrielle, doublée d'un atelier fortement automatisé (section 1.4), qui rend nécessaire un outil de pilotage capable d'agrégier des commandes hétérogènes venues de l'ensemble du groupe.

1.1.1 Quelques repères économiques

Le groupe réalise un chiffre d'affaires de l'ordre de 1,5 à 1,8 million d'euros sur les dernières années (tableau 1.1). Cette trajectoire légèrement orientée à la baisse renforce l'intérêt des projets menés durant ce stage : dans un contexte de maîtrise des coûts, tout gain de productivité et toute réduction des erreurs de saisie dans l'atelier prennent une valeur d'autant plus stratégique.

1.2 Les sites e-commerce du groupe

Le groupe opère plus d'une dizaine de boutiques en ligne, toutes propulsées par WooCommerce, l'extension e-commerce de WordPress (tableau 1.2).

Table 1.1. Chiffre d'affaires du groupe Tikimob (estimations)

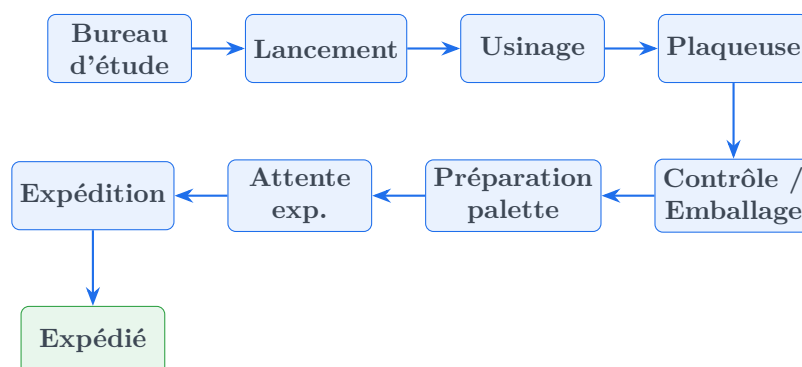
Année	CA (M€)	Évolution
2023	1,8	—
2024	1,6	↓
2025	1,5	↓

Table 1.2. Sites e-commerce du groupe Tikimob

Site	Adresse web	Segment
Tikimob FR	tikimob.fr	Mobilier en kit sur mesure — France
Tikimob ES	tikimob.es	Mobilier en kit sur mesure — Espagne
Tikimob DE	tikimob.de	Mobilier en kit sur mesure — Allemagne
Tikimob IT	tikimob.it	Mobilier en kit sur mesure — Italie
Tikivan	tikivan.fr	Aménagement de vans
Atelier TKM	atelier-tkm.fr	Agencement sur mesure pour professionnels
RenovPlacard	renovplacard.fr	Rénovation de placards
Mobbia	mobbia.fr	Mobilier standard sur mesure éco
Tipano	tipano.fr	Panneaux décoratifs
Fabrique du Carton	lafabriqueducarton.fr	Mobilier en carton sur mesure
Loft Collection	loftcollection.fr	Mobilier standard design
Cuisina9	cuisina9.fr	Cuisines sur mesure
Frontenmeister	frontenmeister.de	Cuisines — Allemagne

1.3 L'atelier de production

L'atelier constitue le cœur opérationnel du groupe. Il est organisé en **postes de travail successifs** formant une chaîne de production qu'il était essentiel de modéliser fidèlement dans l'outil informatique. Le séquençement réel, reconstitué avec les opérateurs, est présenté en figure 1.1.

**Figure 1.1.** Chaîne de production de l'atelier Tikimob

Chaque poste joue un rôle précis :

1. **Bureau d'étude (BE)** : réception de la commande et création des plans techniques du meuble (et non de simples « maquettes » commerciales). C'est l'étape de traduction du besoin client en données fabricables.

2. **Lancement** : contrôle des plans réalisés par le BE, modélisation des panneaux à découper, puis envoi des fichiers vers les commandes numériques de l'atelier. C'est ce poste qui « lance » concrètement la fabrication.
3. **Usinage** : découpe et façonnage des panneaux selon les dimensions issues du configurateur et des plans.
4. **Plaqueuse** : application du plaquage de chant (la bande qui habille la tranche des panneaux) sur les pièces découpées.
5. **Contrôle / Emballage** : vérification qualité de chaque pièce puis conditionnement du meuble en kit.
6. **Préparation palette** : regroupement des colis d'une même commande et montage de la palette.
7. **Attente expédition** : la commande est prête et attend l'enlèvement par le transporteur.
8. **Expédition / Expédié** : départ effectif vers le transporteur, puis clôture de la commande.

Quincaillerie. Certaines commandes contiennent de la quincaillerie (vis, charnières, coulisseries...). Dès le poste de Lancement, l'outil le signale : la commande est dupliquée dans un circuit « Quincaillerie » dédié. Une fois traitée, la copie disparaît automatiquement ; la commande « mère » continue son parcours.

Cartons. Le site « Fabrique du Carton » fabrique du mobilier en carton, qui ne nécessite pas tout le parcours d'usinage. Ces commandes rejoignent directement la fin de l'atelier, à partir du poste « Attente expédition ».

1.4 Un atelier fortement automatisé

La production est concentrée dans une usine **100 % française** et largement **automatisée**, implantée à Périgny, près de La Rochelle. Inspiré des standards de l'**industrie 4.0** et bâti en partenariat avec le fabricant de machines **Biesse**, l'atelier conjugue automatisation et chaîne numérique de bout en bout :

- un **magasin automatisé de panneaux** (Winstore) stocke et gère environ **1 800 panneaux** de bois et alimente directement les machines, ce qui limite les manipulations ;
- un **centre d'usinage 3 axes** découpe et façonne avec une précision millimétrique les éléments conçus en 3D par les clients ;
- chaque panneau est identifié par un **étiquetage à codes-barres**, garantissant la traçabilité des pièces tout au long de la fabrication ;
- enfin, une **machine de fabrication de cartons sur mesure** adapte l'emballage aux dimensions exactes de chaque meuble, sécurisant le transport tout en optimisant les flux logistiques.

C'est précisément ce niveau d'automatisation qui justifiait mes deux missions. Un atelier où les panneaux sont stockés, découpés, étiquetés et emballés par des machines pilotées numériquement ne pouvait pas rester piloté, côté *suivi des commandes*, par un simple fichier Excel ressaisi à la main : le suivi accusait un retard permanent sur la production réelle. La **mission 1** comble cet écart en donnant à la chaîne automatisée un système d'information à sa mesure. La **mission 2** va plus loin : une partie des fichiers produits par l'outil de conversion (mission 2, chapitre 9) alimente **directement les lignes d'emballage et la machine à cartons** décrites ci-dessus — la « feuille à imprimer » pilote ainsi la cartonneuse à partir des dimensions calculées des colis.

1.5 Organisation informatique

L'infrastructure repose entièrement sur l'écosystème **WordPress / WooCommerce**, hébergé chez OVH. Chaque site dispose de sa propre instance WordPress avec le système **HPOS** (High-Performance Order Storage) activé — un mode de stockage récent de WooCommerce qui range les commandes dans des tables dédiées plus performantes, au lieu de la table générique historique `wp_postmeta`.

Un **configurateur 3D** de meuble sur mesure, développé en interne, permet aux clients de paramétrer leur produit en ligne (dimensions, matériaux, finitions) et de le visualiser en trois dimensions. Lorsqu'un client valide sa configuration, les paramètres saisis sont enregistrés dans le nom du produit sous une forme structurée, par exemple « Étagère : modèle A - L=160cm H=200cm P=35cm ». Ce nom « parlant » doit ensuite être **décodé** côté logiciel pour en extraire les dimensions chiffrées exploitables par l'atelier (voir chapitre 5).

L'instance centrale de développement et de test est hébergée sur `tikitest.ovh` ; elle permet de valider chaque évolution avant déploiement sur les sites de production.

Contexte et problématique

2.1 Un cahier des charges à approfondir

À mon arrivée, le besoin m'a été transmis sous la forme d'un cahier des charges lui-même rédigé dans un fichier Excel (`Projet_Outil_suivi.xlsx`). Ce fichier décrivait, onglet par onglet, les colonnes que chaque poste souhaitait suivre. Il constituait un excellent point de départ, mais restait incomplet : une grande partie de mon travail initial a consisté à **approfondir chaque besoin**, en interrogeant tour à tour le bureau d'étude, le lancement et les opérateurs.

De ces échanges sont nés des besoins précis absents du document initial : circuits Quincaillerie et Cartons, gestion des acomptes, exports transporteurs, retours qualité, etc. Le périmètre réel s'est révélé bien plus large que le cahier des charges de départ.

2.2 La situation de départ : le fichier Excel partagé

2.2.1 Structure de l'ancien fichier

L'analyse du fichier d'origine révèle une structure en **sept onglets** : *Feuill* (vue d'ensemble, 24 colonnes), *Bureau* (vue étendue, 40 colonnes), *Usinage* (8 colonnes), *Plaqueuse* (7 colonnes), *Contrôle Emballage* (10 colonnes, dont « pièce à refaire »), *Expédition* (10 colonnes) et un *Modèle de base* (gabarit, 24 colonnes).

2.2.2 Les limites identifiées

1. **Édition impossible à plusieurs.** Une seule personne pouvait modifier le fichier à la fois, créant files d'attente et conflits de versions.
2. **Saisie manuelle.** Chaque commande était recopiée à la main depuis WooCommerce — source d'erreurs et de perte de temps.
3. **Aucune traçabilité.** Impossible de savoir qui avait modifié quoi, et quand.
4. **Pas de contrôle d'accès.** Tout le monde voyait et modifiait tout, y compris des données commerciales sensibles.
5. **Données fragiles.** Risque de perte en cas de corruption ou de fausse manipulation.
6. **Aucune automatisation.** Surfaces, dimensions et agrégations calculées à la main.
7. **Multi-sites impossible.** Le fichier ne couvrait historiquement que Tikimob FR.
8. **Lenteur de recherche.** Retrouver une commande imposait de naviguer dans le WordPress de chaque site, aux temps de chargement longs.

2.3 Estimation du temps perdu

Pour quantifier l'impact, j'ai mené avec mon tuteur un travail d'estimation comparative (tableau 2.1).

Table 2.1. Comparatif des temps : ancien processus vs. plugin Suivi Commandes

Tâche	Excel (min)	Plugin (min)	Gain
Saisie d'une nouvelle commande	10–15	0 (auto)	100 %
Recherche d'une commande	3–5	0,5	~85 %
Mise à jour de l'état (par poste)	8–12	0,5	~95 %
Vérification du poids total	2–3	0 (auto)	100 %
Import de 50 commandes multi-sites	120–180	5–10	~95 %
Gestion d'une non-conformité	5–10	1	~85 %
Total estimé / jour (30 cmd)	~90	~10	~89 %

Le passage du fichier Excel au plugin représente un gain de l'ordre de **80 minutes par jour et par opérateur** sur les tâches de suivi, soit plus de **6 heures par semaine**. Sur une année de 220 jours ouvrés, cela représente environ **290 heures de temps libéré** pour les équipes. C'est le bénéfice central du projet, et c'est lui qui en justifie l'investissement.

2.4 Le cahier des charges consolidé

À partir de l'analyse de l'existant et des besoins exprimés, le cahier des charges suivant a été défini :

Réf.	Exigence
CC1	Centraliser les commandes de tous les sites dans un tableau unique.
CC2	Automatiser l'import via l'API REST WooCommerce.
CC3	Proposer des vues filtrées par poste de travail.
CC4	Implémenter un workflow de production avec progression d'état automatisée.
CC5	Mettre en place un système de rôles (RBAC) limitant l'accès selon le profil.
CC6	Extraire automatiquement dimensions et surfaces depuis les noms de produits.
CC7	Gérer la non-conformité avec retour automatique au poste concerné.
CC8	Permettre la synchronisation bidirectionnelle avec les sites distants.
CC9	S'intégrer nativement à WordPress / WooCommerce sous forme de plugin.
CC10	Fournir un tableau de bord analytique de pilotage.
CC11	Automatiser la génération des fichiers d'usinage et de transport (mission 2).

Architecture technique (Mission 1)

3.1 Vue d'ensemble

Le plugin suit une architecture dite **API-first**, inspirée des applications web modernes appelées **SPA** (Single Page Application, « application monopage »). Concrètement, le serveur (PHP) ne renvoie pas des pages HTML déjà construites : il expose une **API REST**, c'est-à-dire un ensemble d'adresses web (URL) que le navigateur interroge pour lire ou écrire des données au format JSON. Côté navigateur, une application Vue.js consomme cette API et met à jour l'affichage sans jamais recharger toute la page.

Cette séparation nette entre le serveur (les données) et l'interface (l'affichage) présente deux avantages : l'interface est très fluide pour l'opérateur, et le cœur métier (l'API) pourrait demain être réutilisé par un autre front-end, voire par une application mobile, sans réécrire la logique de production. Le flux global est représenté en figure 3.1.

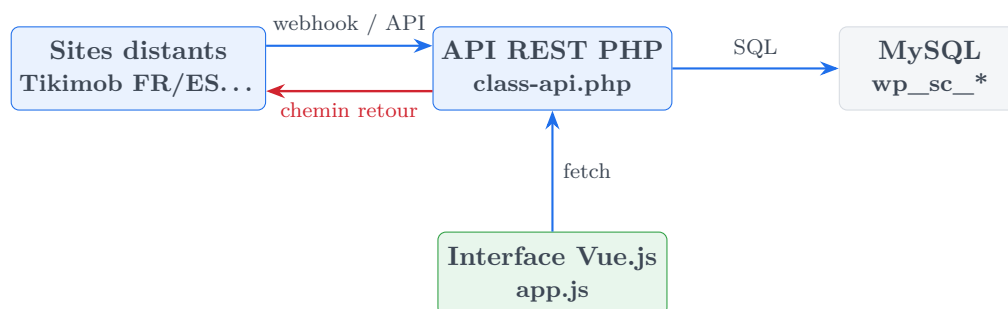


Figure 3.1. Architecture globale du système Suivi Commandes

3.2 Stack technologique

Le terme *stack* (« pile ») désigne l'ensemble des technologies empilées pour faire fonctionner l'application (tableau 3.1).

3.3 Structure des fichiers du plugin

Le plugin sépare l'interface d'administration, les ressources front-end (*assets*) et la logique métier (*includes*). Le projet réel représente aujourd'hui environ **33 000 lignes de code réparties dans 18 fichiers** (tableau 3.2).

Table 3.1. Technologies utilisées

Couche	Technologie	Rôle
Front-end	Vue.js 3	Interface réactive (SPA) du tableau de bord
Visualisation	Chart.js	Graphiques du tableau de bord analytique
Style	CSS3 sur mesure	Thème responsive adapté aux écrans d'atelier
Back-end	PHP 8.0+	API REST, logique métier, hooks WooCommerce
Framework	WordPress 6.0+	CMS hôte, gestion des utilisateurs
E-commerce	WooCommerce + HPOS	Source des commandes, API REST v3
Base de données	MySQL / MariaDB	Stockage des commandes et du suivi
File d'attente	WP-Cron	Reprise auto. des envois en erreur (chemin retour)
Champs perso.	Advanced Custom Fields	Données de plans côté distant
Hébergement	OVH	Serveur mutualisé

3.4 Modèle de données

3.4.1 Schéma de la base

Le plugin crée **18 tables personnalisées** (toutes préfixées `wp_sc_`). La table maîtresse est `wp_sc_commandes`, entourée de tables satellites (tableau 3.3).

3.4.2 Choix de modélisation

Surface par article, pas par commande. La surface (`surface_m2`) est stockée individuellement pour chaque meuble dans `wp_sc_commande_articles`, à partir de ses dimensions largeur × hauteur. Additionner les surfaces n'aurait aucun sens, car des meubles différents utilisent des panneaux de matières et d'épaisseurs différentes.

Statut WooCommerce vs. état de production. Le champ `statut_wc` stocke, en lecture seule, le statut commercial réel sur WooCommerce. Le champ `etat` représente la position dans le pipeline (un type SQL ENUM restreignant les valeurs à une liste figée). Ces deux notions sont volontairement séparées.

Une ligne de suivi par poste. Chaque table de suivi possède une contrainte d'unicité sur `commande_id`, garantissant une relation « une commande, une ligne de suivi » par poste.

Table 3.2. Principaux fichiers du plugin

Fichier	Rôle
suivi-commandes.php	Point d'entrée : déclare le plugin, charge les modules, programme les crons.
includes/class-database.php	Création des 18 tables et migrations automatiques.
includes/class-api.php	Cœur de l'API REST : 60 routes (plus gros fichier back-end).
includes/class-roles.php	Système de rôles et permissions (RBAC).
includes/class-status-mapper.php	Traduction des statuts WooCommerce en états internes.
includes/class-woocommerce.php	Interception des commandes, extraction dimensions/poids/DSU.
includes/class-remote-import.php	Import multi-sites depuis les API WooCommerce distantes.
includes/class-remote-push.php	Chemin retour : renvoi des données vers les sites d'origine.
includes/class-archive.php	Archivage automatique des commandes terminées.
includes/class-refresh-cron.php	Actualisation quotidienne des commandes actives.
includes/class-export-engine.php	Moteur d'exports industriels (EDI, Sormec...) — mission 2.
includes/class-panneaux-referentiel.php	Référentiel matières et poids des panneaux — mission 2.
includes/class-xlsx-writer.php	Génération de fichiers Excel colorés — mission 2.
admin/class-admin.php	Pages d'administration WordPress.
assets/js/app.js	Application Vue.js complète, ~10 500 lignes.
assets/css/style.css	Feuille de style, ~4 700 lignes.

Table 3.3. Tables principales de la base de données

Table	Contenu
wp_sc_commandes	Table maîtresse : une ligne par commande (client, site, état, montants, DSU...).
wp_sc_commande_articles	Un article (meuble) par ligne, avec ses dimensions et sa surface.
wp_sc_suivi_bureau_etude / lancement	Suivi des deux premiers postes.
wp_sc_suivi_usinage / plaqueuse / quincaillerie	Suivi des postes de fabrication.
wp_sc_suivi_controle / emballage / carton	Contrôle qualité, emballage, circuit carton.
wp_sc_suivi_attente_exp	Préparation palette et attente d'expédition.
wp_sc_pieces_refaire	Journal des pièces signalées non conformes.
wp_sc_retours_atelier	Tickets SAV et retours ciblés vers un poste.
wp_sc_etat_history	Historique horodaté des changements d'état.
wp_sc_notifications	Notifications internes.
wp_sc_push_queue	File d'attente du chemin retour (envois à retenter).
wp_sc_commandes_archive	Commandes expédiées de plus de 30 jours.
wp_sc_commande_order	Ordre d'affichage personnalisé par poste.
wp_sc_sites_origine	Référentiel des sites distants et de leurs accès.

Développement front-end (Mission 1)

4.1 L'application Vue.js

L'interface est une application monopage (SPA) écrite avec **Vue.js 3**, préféré à React pour sa légèreté, sa réactivité et sa facilité d'intégration dans WordPress, sans étape de compilation complexe. « Réactif » signifie ici que, dès qu'une donnée change (un opérateur valide une étape), l'affichage se met à jour tout seul, sans recharger la page.

Toute l'application tient dans un unique fichier `app.js` (~10 500 lignes), chargé par WordPress avec un système de versioning dynamique destiné à forcer le navigateur à recharger la dernière version après chaque déploiement.

```
1 wp_enqueue_script(  
2   'sc-app',  
3   SC_PLUGIN_URL . 'assets/js/app.js',  
4   [],  
5   time(), // change a chaque chargement => force le rechargement  
6   true  
7 );
```

Listing 4.1. Chargement de l'application avec anti-cache

4.2 Des vues adaptées à chaque poste

L'interface affiche un contenu différent selon le rôle de l'utilisateur connecté : chaque opérateur ne voit que les colonnes et actions qui le concernent. Dès la connexion, un routage automatique l'amène sur sa vue dédiée (tableau 4.1).

4.3 Fonctionnalités d'interface avancées

Édition en ligne (double-clic). Les utilisateurs autorisés modifient une donnée directement en double-cliquant sur la cellule, sans passer par un formulaire séparé. Un formulaire modal « Modifier » reste disponible pour les éditions complètes.

Tri et filtrage. Le tableau propose un tri côté navigateur, par défaut par ancienneté (logique FIFO de l'atelier). Les colonnes les plus utiles (Client, Date, Pays, n° de commande) sont en tête. Des filtres permettent de n'afficher que certains statuts ou sites.

Badges de statut colorés. Les statuts WooCommerce sont affichés en pastilles colorées en lecture seule, distinctes de l'état de production : vert pour « en cours », orange pour « en attente de paiement », jaune pour « en attente ».

Table 4.1. Vues par rôle utilisateur

Vue	Colonnes principales	Actions
Bureau (global)	Toutes les colonnes	Édition complète
Bureau d'étude	Plans techniques, opérateur, temps	Saisie plans, valider
Lancement	Plans, panneaux, quincaillerie	Contrôle plans, lancer
Usinage	Nom, n° cmd, DSU, configurateur	Temps, commentaire, valider
Plaqueuse	Nom, n° cmd, DSU, configurateur	Temps, commentaire, valider
Quincaillerie	Commandes avec quincaillerie	Préparer (la copie disparaît)
Contrôle / Emballage	Nom, n° cmd, DSU, configurateur	Contrôle, pièce à refaire
Préparation / Attente exp.	Poids, transporteur, palette	Monter palette, déclarer prête
Expédition	Poids, transporteur, n° cmd	Saisir transporteur, expédier
Carton	Commandes mobilier carton	Traiter, envoyer en attente exp.
Comptable	Montants, acomptes/soldes	Lecture des données financières

Les lignes dont la **DSU** (Date de Sortie d'Usine) est dépassée sont surlignées en **rouge**, et un compteur d'urgences s'affiche en pied de tableau. Cette mise en évidence visuelle garantit qu'un retard de production saute immédiatement aux yeux de l'opérateur.

Développement back-end (Mission 1)

5.1 Une API REST sur mesure

Le back-end expose une API REST complète sous le *namespace* (préfixe d'URL) `suivi-commandes/v1`, avec un alias `sc/v1` pour compatibilité. L'API compte aujourd'hui **60 routes**. Une « route » est une adresse associée à une action (tableau 5.1).

Table 5.1. Échantillon des routes de l'API REST (60 au total)

Méthode	Route	Description
GET	<code>/commandes</code>	Liste toutes les commandes
GET	<code>/commandes/{id}</code>	Détail d'une commande
PUT	<code>/commandes/{id}</code>	Mise à jour générale
POST	<code>/commandes/{id}/advance</code>	Avancement d'état au poste suivant
POST	<code>/commandes/{id}/push</code>	Chemin retour vers le site d'origine
POST	<code>/commandes/{id}/refresh-from-remote</code>	Réactualise depuis le site
POST	<code>/commandes/{id}/ticket-commande</code>	Crée un ticket SAV
POST	<code>/commandes/{id}/export-edi-vir</code>	Fichier transporteur EDI VIR
POST	<code>/commandes/{id}/export-conversion</code>	Conversion d'usinage (mission 2)
GET	<code>/stats/dashboard</code>	Données du tableau de bord
GET	<code>/stats/operateurs</code>	Statistiques par opérateur
POST/GET	<code>/webhook/commande</code>	Réception d'un webhook distant
DELETE	<code>/commandes/{id}</code>	Suppression (admin)

5.2 Intégration avec WooCommerce

5.2.1 Les hooks natifs

Un *hook* (« crochet ») est un mécanisme de WordPress permettant d'exécuter son propre code en réaction à un événement. Le plugin s'accroche à deux événements WooCommerce.

```

1 // Interception de toute nouvelle commande
2 add_action('woocommerce_new_order',
3     [SC_WooCommerce::class, 'on_new_order'], 10, 1);
4
5 // Suivi des changements de statut
6 add_action('woocommerce_order_status_changed',
7     [SC_WooCommerce::class, 'on_status_change'], 10, 4);

```

Listing 5.1. Branchement sur les événements WooCommerce

5.2.2 Le décodage des dimensions

Le défi technique le plus marquant a été l'extraction automatique des dimensions des meubles à partir des noms de produits. Ces noms suivent des formats variés : verbeux (« L=160 cm H=200 cm P=35 cm »), compact (« 70x35x15cm ») ou mixte. La solution repose sur les **expressions régulières** — un langage de motifs permettant de repérer une suite de caractères précise. Le code essaie d'abord le format verbeux, puis se rabat (*fallback*) sur le format compact.

```

1 // Format verbeux : L=...cm H=...cm
2 if (preg_match('/L=([0-9.]+)\s*cm\s*H=([0-9.]+)\s*cm/i', $texte, $m)) {
3     $largeur = (float) str_replace(',', '.', $m[1]);
4     $hauteur = (float) str_replace(',', '.', $m[2]);
5 }
6 // Repli : format 70x35(x15)cm
7 elseif (preg_match('/(\d+)\s*[xX]\s*(\d+)/i', $texte, $m)) {
8     $largeur = (float) $m[1]; $hauteur = (float) $m[2];
9 }
10 // Surface frontale en m2
11 $surface = round(($largeur * $hauteur) / 10000, 4);

```

Listing 5.2. Extraction multi-format des dimensions

5.2.3 Le calcul de la DSU

La **DSU** initiale est calculée automatiquement à l'import à partir de la date de commande, en ajoutant un délai en jours ouvrés. Un champ DSU « new » reste modifiable manuellement par le bureau d'étude si le délai change.

5.3 La traduction des statuts (status-mapper)

Chaque site utilise ses propres libellés de statut WooCommerce, souvent très spécifiques (**placage-chants**, **prepa-quincaillerie**, **transport-en-cour**...). Le module `class-status-mapper.php` centralise une grande table de correspondance qui traduit chacun de ces dizaines de statuts en l'un des états internes du pipeline. Par exemple, **placage-chants** devient **plaqueuse**, **prepa-quincaillerie** devient **quincaillerie**, et la quarantaine de statuts liés au transport sont ramenés à **expedition**. C'est ce module qui unifie des sites hétérogènes derrière un workflow commun.

5.4 Le plugin compagnon `sc-expose-meta.php`

Pour enrichir les données accessibles via l'API WooCommerce des sites distants, un petit plugin compagnon (`sc-expose-meta.php`) est déployé sur chaque site. Il rend disponibles, dans la réponse de l'API, des informations métier (poids total, DSU, données de plans) absentes par défaut. Il apporte trois améliorations :

- **Un mode allégé pour aller plus vite.** Lorsque le plugin central n'a besoin que de l'identifiant des commandes (scan initial, chapitre 6), le compagnon le détecte et n'effectue aucun traitement supplémentaire. Sur 100 commandes, cela évite une centaine de calculs inutiles.

- **La bonne version de plan.** Lorsqu'un plan est refusé puis refait (v2, v3...), c'est la dernière version qui doit remonter. Le compagnon retient toujours la version la plus récente.
- **Affichage de la bonne valeur.** Une correction a réglé un cas où une référence technique interne s'affichait à la place de la valeur réelle dans la colonne « Version de plan ».

5.5 Le système de rôles (RBAC)

Le plugin implémente un contrôle d'accès basé sur les rôles (**RBAC**, Role-Based Access Control), en s'appuyant sur le système natif de WordPress (les *capabilities*). Le système compte une vingtaine de rôles, des administrateurs aux opérateurs mono-poste, en passant par des rôles combinés et un rôle Comptable. Chaque permission (`sc_view_all`, `sc_edit_usinage`, `sc_delete...`) est **vérifiée côté API** avant toute opération (tableau 5.2).

Table 5.2. Aperçu des rôles RBAC (une vingtaine au total)

Rôle	Permissions principales
Super Admin	Tout : vue globale, édition tous postes, suppression, import, gestion utilisateurs.
Admin Commerce	Vue globale, statistiques, édition des données commerciales.
Admin Production	Vue globale, statistiques, édition de tous les postes.
Bureau d'étude + Lancement	Vue et édition des deux premiers postes + suivi global.
Opérateur mono-poste	Vue et édition d'un seul poste (usinage, plaqueuse...).
Atelier multi-postes	Contrôle, emballage et attente expédition réunis.
Carton	Circuit carton + emballage.
Comptable	Lecture des données financières (acomptes, soldes, montants).

Import multi-sites et chemin retour (Mission 1)

6.1 Le défi de l'agrégation multi-sites

L'un des apports majeurs du plugin est sa capacité à agréger automatiquement, dans un tableau unique, les commandes de tous les sites du groupe. Le plugin gère **treize sites distants**, avec des mécanismes d'authentification différenciés (clés API WooCommerce pour la majorité, authentification basique pour certains).

6.2 Le mécanisme d'import en trois phases

Phase 1 — scan des identifiants. Le plugin envoie d'abord une requête légère à l'API distante avec `_fields=id` : il ne récupère que les identifiants. Grâce au court-circuit du plugin compagnon, cette requête est extrêmement rapide.

Phase 2 — import par lot (batch). Le plugin utilise ensuite le paramètre `include=` de l'API WooCommerce pour récupérer plusieurs commandes en un seul appel réseau. C'est l'optimisation majeure : au lieu de 20 appels de 5 s (100 s), un seul appel groupé de 8 s suffit. Pour résister aux sites lents, l'import applique une **stratégie dégradée à trois niveaux** :

1. **Complet** (timeout 60 s) : tous les champs, métadonnées et détail des articles.
2. **Sans métadonnées** (timeout 45 s) : on garde les articles, on abandonne les métadonnées.
3. **Minimal** (timeout 30 s) : uniquement les données de base (statut, dates, client).

Si un niveau échoue par dépassement de délai, le plugin réessaie automatiquement avec le niveau suivant. On garantit ainsi qu'au moins une partie des données est toujours récupérée.

Phase 3 — insertion et enrichissement. Les données reçues sont traitées localement : décodage des dimensions, calcul des surfaces, dédoublonnage (par couple `wc_order_id` + `site_origine`), classification de l'état via le status-mapper, puis insertion en base.

6.3 Le chemin retour (synchronisation bidirectionnelle)

Présenté comme « à venir » dans la première version du rapport, le chemin retour est aujourd'hui **développé, déployé progressivement** (d'abord sur le site Mobbia) **et opérationnel**. Il referme la boucle : non seulement le plugin importe les commandes, mais il

renvoie aussi vers le site d'origine les modifications faites dans l'atelier (changement de DSU, notes, nouveau statut, version de plan validée).

Le module `class-remote-push.php` (~1 300 lignes) en porte toute la logique. Pour une commande donnée :

1. Construction d'une charge utile (*payload*) à partir de la commande locale, via une table de correspondance champ par champ.
2. Traduction de l'état interne en statut WooCommerce attendu par ce site précis (chaque site a sa propre table de statuts, configurable).
3. **Détection de conflit** : avant d'écrire, le plugin compare la date de dernière modification du site distant à celle qu'il avait mémorisée. Si le site a changé entre-temps, il signale un conflit plutôt que d'écraser aveuglément.
4. Envoi via l'API REST WooCommerce du site distant.
5. En cas d'échec réseau ou serveur (5xx), la commande est placée dans une file d'attente persistante (`wp_sc_push_queue`) ; un cron la réessaie. En cas d'erreur définitive (4xx), elle est marquée en échec sans réessai.

Un raffinement notable : si l'envoi échoue spécifiquement à cause du statut, le plugin retente automatiquement le même envoi **sans le statut**, afin de sauver au moins la mise à jour des autres données plutôt que de tout perdre.

Workflow de production (Mission 1)

7.1 La machine à états

Le cœur du plugin est une **machine à états** (*state machine*) : un modèle qui décrit l'ensemble des états possibles d'une commande et les transitions autorisées. Une commande ne peut pas sauter d'étape ni revenir en arrière arbitrairement ; elle suit le chemin défini (figure 7.1), à l'exception d'une boucle qualité.

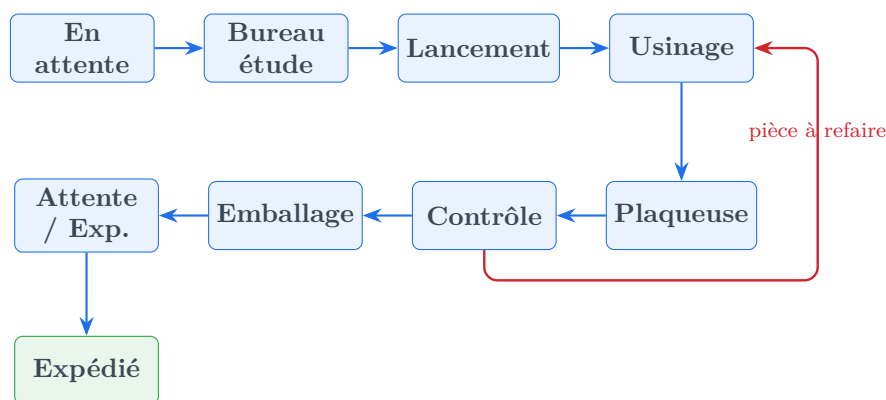


Figure 7.1. Machine à états du workflow de production (avec boucle qualité)

7.2 La gestion de la non-conformité

Lorsque le poste Contrôle signale une « pièce à refaire », le système :

1. ramène automatiquement la commande au poste de fabrication concerné ;
2. déclenche une alerte visuelle rouge de haute priorité et une notification sur le poste cible ;
3. conserve la trace du signalement dans `wp_sc_pieces_refaire` et les commentaires associés.

Ce mécanisme de **boucle qualité** garantit qu'**aucune pièce non conforme ne peut atteindre l'expédition** sans avoir été retravaillée puis re-contrôlée. C'est une règle métier non négociable, imposée côté serveur.

7.3 L'avancement automatique

La route `POST /commandes/{id}/advance` permet à un opérateur de valider la fin de son intervention et de faire progresser la commande vers le poste suivant. Avant d'autoriser l'avancement, le

plugin vérifie que l'utilisateur possède bien la permission du poste courant : un opérateur plaqueuse ne peut pas faire avancer une commande encore en usinage. Chaque transition est horodatée dans `wp_sc_etat_history`.

Modules avancés (Mission 1)

Au-delà du socle décrit jusqu'ici, le plugin s'est enrichi de plusieurs modules qui n'étaient pas tous prévus au départ mais qui ont répondu à des besoins concrets remontés du terrain.

8.1 Tableau de bord analytique

Un tableau de bord (routes `/stats/dashboard` et `/stats/operateurs`) présente des indicateurs construits avec **Chart.js** : nombre de commandes par état, temps moyens par poste, charge par opérateur, taux de non-conformité, comparaisons d'un mois sur l'autre. Il s'appuie sur le journal `wp_sc_etat_history`.

8.2 Archivage automatique

Pour ne pas surcharger la vue active, un cron quotidien (`class-archive.php`) déplace dans `wp_sc_commandes_archive` les commandes expédiées depuis plus de 30 jours. Un archivage manuel est aussi possible, commande par commande ou en lot.

8.3 Actualisation quotidienne

Un second jeu de crons (`class-refresh-cron.php`) réinterroge les sites distants deux fois par jour (10 h 30 et 13 h 10) pour rafraîchir les commandes encore actives.

8.4 Tickets SAV et retours atelier

La route `/commandes/{id}/ticket-commande` et la table `wp_sc_retours_atelier` permettent de créer un ticket de service après-vente et de renvoyer une commande déjà avancée vers n'importe quel poste, avec un motif.

8.5 Paiement en plusieurs fois : acompte et solde

Certaines commandes sont réglées en deux fois, ce qui crée sur WooCommerce une commande « acompte » puis une commande « Solde commande #XXXX ». Sans traitement, ce solde apparaissait comme un **doublon parasite**. Le plugin repère désormais ces lignes : il marque la commande de solde, la rattache à la vraie commande (numéro extrait par expression régulière) et signale qu'un acompte a été versé. Une migration rétroactive a nettoyé les commandes déjà en base.

8.6 Notifications internes et routes de diagnostic

La table `wp_sc_notifications` alimente un centre de notifications (arrivée d'une commande à un poste, pièce à refaire, etc.). Par ailleurs, plusieurs routes techniques (`sc-health`, `sc-debug-commandes`, `sc-remap-etats`, `sc-reset-acf`) m'ont servi à diagnostiquer et réparer la base en production sans intervention manuelle directe sur le serveur.

Mission 2 — Outil de calcul et de conversion transport

9.1 Contexte de la mission 2

La seconde partie de mon stage concernait un tout autre outil : un classeur Excel appelé « Outil de conversion transport » (`Outil_de_Conversion_transport.xlsx`). L'extension `.xlsx` indique un fichier Excel contenant des **macros**, c'est-à-dire du code automatisant des actions, écrit en **VBA** (Visual Basic for Applications), le langage intégré à Excel.

Ce classeur était l'outil quotidien de préparation des fichiers de fabrication et d'expédition. Il partait d'un fichier brut exporté par le logiciel de conception « Vision » et, par une série de boutons, le transformait en plusieurs fichiers exploitables : fichier d'usinage, fichiers transporteurs, feuille à imprimer, étiquettes, bon de livraison. Il contenait environ **3 750 lignes de VBA** réparties en deux gros modules.

Ma mission a comporté trois temps : **(1)** comprendre cette logique métier en lisant tout le code, **(2)** la documenter, et **(3)** la ré-implémenter en PHP pour l'intégrer directement au plugin « Suivi Commandes » — afin qu'à terme, un opérateur n'ait plus besoin d'ouvrir Excel.

La figure 9.1 illustre cette transformation : à gauche, l'ancien classeur Excel (le point de départ) ; à droite, la page « Calcul collage » désormais intégrée au plugin (le point d'arrivée).

9.2 Les paramètres de l'outil

L'onglet « Accueil » du classeur rassemble les paramètres saisis avant la conversion (tableau 9.1). On les retrouve, à droite de la figure 9.1, dans le bloc « Paramètres atelier » de la page intégrée — désormais mémorisés et pré-remplis.

9.3 La logique métier reconstituée

9.3.1 Le référentiel des poids de panneaux

L'onglet « Table Poids » associe à chaque référence de matière une épaisseur et un poids au mètre carré (tableau 9.2). C'est la table qui permet de calculer le poids réel d'un panneau à partir de ses seules dimensions : $\text{poids} = \text{surface (m}^2\text{)} \times \text{poids au m}^2$. J'ai repris ce référentiel dans `class-panneaux-referentiel.php`.

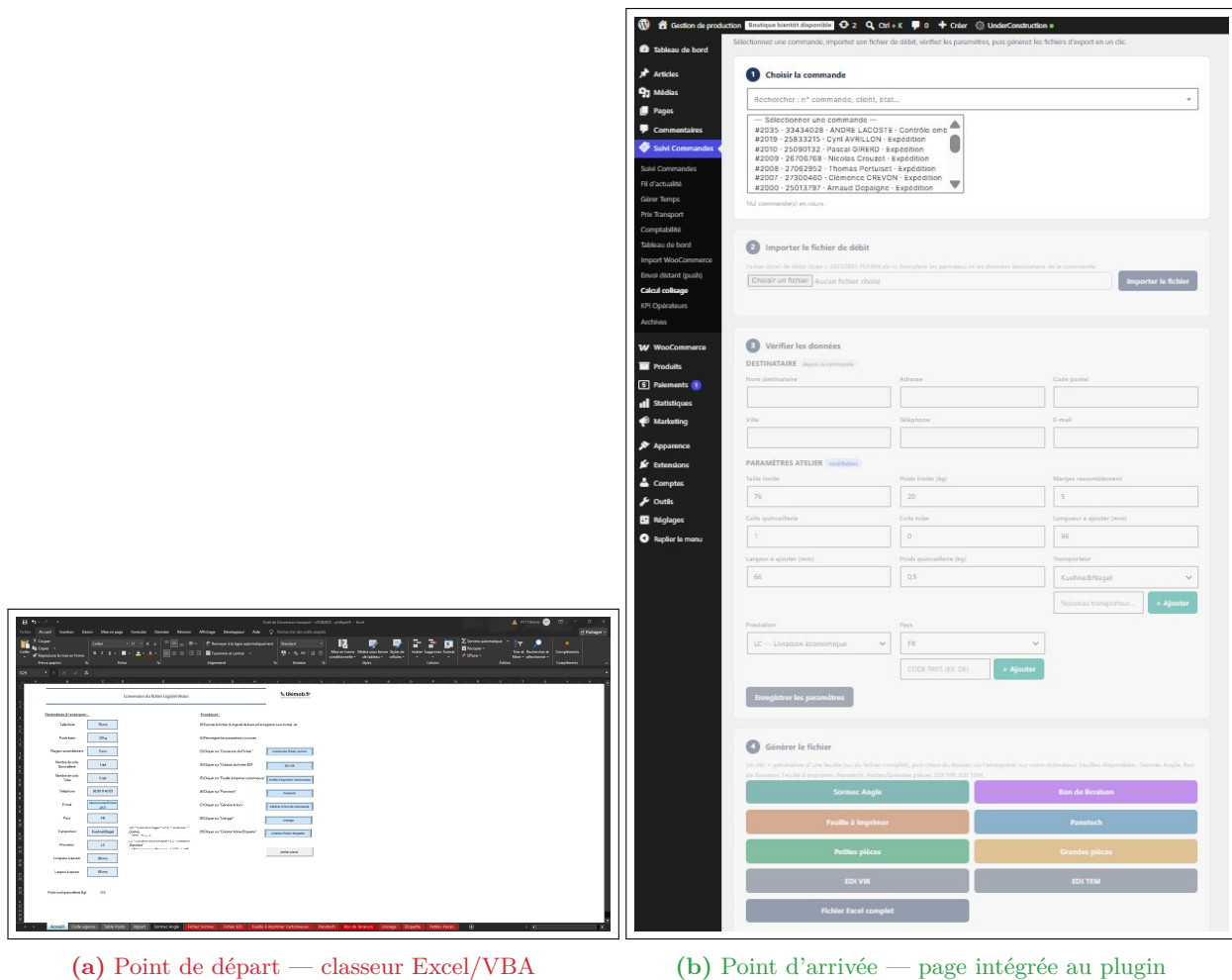


Figure 9.1. De l'outil Excel « Conversion transport » (a) à la page « Calcul collage » du plugin Suivi Commandes (b). Les paramètres saisis à la main et les boutons VBA de gauche sont remplacés, à droite, par un formulaire pré-rempli depuis la base de données et des boutons d'export en un clic.

Les valeurs de poids au mètre carré, qui relèvent du **savoir-faire de l'entreprise**, sont volontairement masquées dans ce rapport.

9.3.2 Le routage transporteur par département

L'onglet « Code agence » fait correspondre à chaque département français (les deux premiers chiffres du code postal) une agence de transporteur. Selon le transporteur choisi, le département de destination détermine le code de centre à inscrire dans le fichier : certains transporteurs utilisent un code unique pour toute la France, d'autres un code différent selon les zones. Cette table département → agence a été reportée en PHP (fonction `code_agence()`).

9.3.3 La conversion du fichier brut

La macro centrale, `Conversion_fichier`, réalise une longue chaîne d'opérations sur le fichier exporté par Vision. Reformulée en langage clair, elle :

Table 9.1. Principaux paramètres de l'onglet Accueil

Paramètre	Valeur type	Signification
Taille limite	76	Au-delà, une pièce est « grande » (logistique spéciale).
Poids limite	20	Poids (kg) au-delà duquel un colis bascule en catégorie supérieure.
Marge de rassemblement	5	Tolérance (cm) pour regrouper des pièces proches.
Nb colis quincaillerie	1	Nombre de colis dédiés à la quincaillerie.
Nb colis tube	0	Nombre de colis dédiés aux tubes.
Longueur / Largeur à ajouter	66 / 66	Marges (mm) pour dimensionner le colis.
Transporteur (au choix)	—	Plusieurs transporteurs partenaires selon la destination.
Prestation	LC	LC économique, LS standard, LM avec montage, etc.

Table 9.2. Structure du référentiel matières (valeurs masquées)

Référence	Épaisseur (mm)	Poids au m ² (kg)
Mélaminé 16 mm	16	<i>masqué</i>
Mélaminé 8 mm	8	<i>masqué</i>
Mélaminé 19 mm	19	<i>masqué</i>
Mélaminé 28 mm	28	<i>masqué</i>
Contreplaqué 18 mm	18	<i>masqué</i>
MDF 19 mm	19	<i>masqué</i>
Compact 8 mm	8	<i>masqué</i>
Plexiglas 5 mm	5	<i>masqué</i>

1. nettoie tous les onglets de travail (Import, Base, Fichier Retravaillé, Sormec, EDI, Panotech, Usinage, Étiquette) ;
2. demande à l'utilisateur de sélectionner le fichier `.xls` à convertir, puis en copie le contenu en valeurs ;
3. réorganise et découpe certaines colonnes (les dimensions, collées dans une seule cellule, sont éclatées en colonnes distinctes) ;
4. normalise l'orientation de chaque pièce : si la longueur est inférieure à la largeur, les deux valeurs sont échangées, pour que « longueur » soit toujours le plus grand côté — condition indispensable au calcul correct des colis ;
5. recopie et structure les données dans l'onglet « Base », socle de toutes les sorties.

9.3.4 Le regroupement des pièces (le « calcul collision »)

C'est le cœur logistique de l'outil. L'objectif est de répartir les pièces en colis cohérents : séparer les grandes pièces, qui voyagent à part, des petites, qui peuvent être regroupées. La macro `RegrouperPiecesGrandes` parcourt chaque ligne et applique un critère de taille.

```

1 For i = 2 To lastRow
2     longueur = wsSrc.Cells(i, "J").Value

```

```

3      largeur  = wsSrc.Cells(i, "L").Value
4      ' Petite piece ? longueur < 350 OU largeur < 200
5      If longueur < 350 Or largeur < 200 Then
6          wsSrc.Rows(i).Copy Destination:=wsDest.Rows(destRow)
7          destRow = destRow + 1
8      End If
9      Next i

```

Listing 9.1. Critère de séparation petites / grandes pièces (VBA d'origine)

Côté plugin, cette logique a été repensée pour produire directement des colis : à partir des dimensions et du poids de chaque pièce, le moteur marie les pièces compatibles dans un même colis tant que le poids limite (20 kg) et les marges de regroupement le permettent, ajoute les marges de longueur et de largeur (66 mm), et calcule le poids brut total. Le poids unitaire provient du référentiel.

9.4 Les fichiers produits

À partir du même fichier d'entrée, l'outil génère plusieurs livrables. Chacun correspondait à un bouton du classeur et à une macro VBA, désormais portés dans le plugin — visibles dans le bloc « Générer le fichier » de la figure 9.1b (tableau 9.3).

Table 9.3. Fichiers de sortie produits par l'outil

Sortie	Usage
Fichier EDI	Fichier transporteur (échange de données informatisé).
Sormec Angle	Fichier d'usinage pour les machines Sormec (pièces d'angle).
Feuille Panotech (.txt)	Fichier texte pour le centre d'usinage Panotech.
Grandes pièces	Liste des pièces volumineuses, traitées et expédiées à part.
Petites pièces	Liste des pièces regroupables en colis.
Feuille à imprimer	Feuille pilotant la machine à carton (dimensions des colis).
Bon de livraison	Document accompagnant la marchandise.
Étiquettes colis	Étiquettes apposées sur chaque colis.

L'intégration au plugin représente un gros gain de temps pour l'atelier. Surtout, les opérateurs n'ont plus à ressaisir à la main les données du client (nom, adresse, coordonnées) sur chaque fichier de sortie : ces informations sont **récupérées automatiquement depuis la base de données** du suivi des commandes. On supprime ainsi à la fois une tâche fastidieuse et une source fréquente d'erreurs de recopie. Plus encore, une partie de ces fichiers ne sert pas seulement à l'humain : elle **alimente directement les équipements automatisés de l'atelier** (section 1.4), notamment les lignes d'emballage et la machine à cartons sur mesure, que la « feuille à imprimer » pilote à partir des dimensions de colis calculées.

9.5 Le portage en PHP et l'intégration au plugin

Reproduire en PHP un traitement Excel pose une difficulté de fond : en VBA, le code manipule directement des cellules (« copier la colonne C dans la colonne AB »), ce qui est illisible et fragile. J'ai donc dû extraire l'**intention métier** derrière chaque manipulation, puis la réécrire sous forme de fonctions claires : `trouver_poids_matiere()`, `calculer_poids_panneau()`, `code_agence()`, `charger_commande_complete()`, etc. Le résultat vit dans trois modules : `class-export-engine.php` (le moteur, ~1 600 lignes), `class-panneaux-referentiel.php` et `class-xlsx-writer.php` (génération de fichiers Excel colorés sans dépendance externe).

Une page d'administration dédiée (« Calcul », figure 9.1b) reproduit chaque étape du classeur sous forme de blocs : l'opérateur choisit une commande, puis clique sur « Lancer » à l'étape voulue. Ces boutons appellent les mêmes routes REST que la page principale, ce qui évite toute duplication de code.

À terme, la fusion des deux missions supprime un aller-retour quotidien fastidieux : plus besoin d'exporter depuis Vision, d'ouvrir le bon classeur Excel, de renseigner les paramètres à la main puis de cliquer sur huit boutons dans le bon ordre. **Tout se déclenche depuis la commande**, dans le même outil que le suivi de production, avec les paramètres déjà mémorisés.

Difficultés rencontrées et solutions

Aucun projet de cette ampleur ne se déroule sans accroc. Ce chapitre présente honnêtement les principaux problèmes rencontrés et les solutions apportées.

10.1 Problèmes de données et d'intégration

Les difficultés les plus formatrices n'ont pas été les bugs ponctuels, mais les **problèmes de cohérence de données** entre treize sites conçus indépendamment. Chaque boutique ayant évolué à son rythme, les mêmes informations n'y portaient pas le même nom, ne suivaient pas le même format, et n'étaient pas toujours présentes. Réconcilier ces sources hétérogènes dans un modèle unique a représenté une part importante du travail.

Le cas le plus représentatif a été celui des **statuts de commande**. Il a fallu construire, en dialoguant avec les équipes, une table de correspondance complète ramenant cette diversité vers les quelques états de production communs — tout en prévoyant un comportement par défaut prudent pour les statuts inconnus, afin qu'une commande ne disparaisse jamais silencieusement du tableau.

Un autre problème, particulièrement insidieux, concernait des **erreurs silencieuses** : certaines insertions en base échouaient sans message, faute d'une parfaite correspondance entre les données reçues et la structure des tables. La solution a été double : fiabiliser le schéma au moyen de migrations automatiques qui vérifient et complètent la structure à chaque mise à jour, et instrumenter systématiquement les opérations sensibles par des journaux, afin de rendre visible ce qui était jusque-là invisible.

10.2 Problèmes d'interface

Boutons masqués (z-index). L'en-tête du tableau, rendu « collant » (sticky), recouvrait les boutons d'action de la première ligne et interceptait les clics. La solution : appliquer `pointer-events: none` sur l'en-tête tout en préservant l'interactivité de ses cellules via `pointer-events: auto`.

En JavaScript, la chaîne « 0 » est considérée comme *falsy*, ce qui faussait l'affichage de certains badges quand la valeur valait zéro. La correction a imposé une comparaison stricte (`===`). Ce bug est d'autant plus piégeux que « 0 » est « vrai » dans la plupart des autres langages.

Bouton de suppression invisible. Une incohérence dans la façon dont les permissions étaient transmises entre le serveur et l'interface rendait le bouton de suppression invisible pour les administrateurs. Uniformiser ce format côté API a résolu le problème.

10.3 Problèmes de déploiement

Un problème récurrent a été le **rechargement accidentel d'anciennes versions** du ZIP, donnant l'impression que les correctifs n'avaient pas pris. La solution a été d'afficher à l'écran un marqueur de version (la constante `SC_VERSION`) : une capture d'écran suffit alors à confirmer la version en production.

10.4 La difficulté propre à la mission 2

Le portage du VBA a présenté sa propre difficulté : le code Excel **mélange logique métier et manipulation de cellules**. Comprendre « ce que fait vraiment » une macro a demandé une lecture patiente, ligne à ligne, pour distinguer l'intention (calculer un poids, regrouper des pièces) de l'implémentation. C'est cette traduction d'intention, plus que la simple réécriture, qui a constitué l'essentiel du travail.

Surtout, cette mission a reposé sur **beaucoup d'échanges** avec les personnes qui utilisent quotidiennement l'ancien fichier Excel. Le code seul ne suffisait pas : il fallait comprendre à quoi correspondait chaque colonne des fichiers de sortie, pourquoi telle valeur était attendue à tel endroit par une machine ou un transporteur, et quelles habitudes de travail s'étaient construites autour de l'outil.

Bilan technique et perspectives

11.1 Bilan technique

Le plugin « Suivi Commandes » (version 6.16.0), accompagné du compagnon `sc-expose-meta.php` (v2.2), répond aujourd'hui à **l'intégralité du cahier des charges** (CC1 à CC11), largement enrichi en cours de route. Les onze exigences initiales — de la centralisation multi-sites au workflow de production, en passant par le système de rôles, la synchronisation bidirectionnelle, le tableau de bord analytique et les exports industriels de la mission 2 — sont désormais développées, déployées et utilisées en production. Le périmètre réellement livré dépasse même le cahier des charges de départ, avec l'ajout des tickets SAV, de la gestion des paiements en plusieurs fois et de l'archivage automatique.

11.1.1 Documentation et passation

Avant la fin de mon stage, j'ai rédigé un **manuel de maintenance et de reprise** complet (~18 pages) destiné à la personne qui reprendra le projet après mon départ. Ce document guide le lecteur **pas à pas** : arborescence du code et rôle de chaque classe, procédure de mise à jour, ajout d'un site source ou d'un nouveau poste d'atelier, gestion des rôles, tableau de dépannage, sauvegarde / restauration et procédures d'urgence. L'objectif était qu'un développeur découvrant le plugin puisse intervenir sereinement, sans dépendre d'une transmission orale, et que le travail réalisé reste **pérenne au-delà de mon passage**.

11.2 Ce qu'il reste à faire

Court terme : généraliser le déploiement. La mission 2 est fonctionnelle et intégrée au plugin ; il reste à l'activer sur l'ensemble des sites et à retirer définitivement l'ancien classeur Excel de la chaîne quotidienne, après une courte période de double fonctionnement servant de validation croisée.

Long terme : fonctionnalités avancées.

- Notifications temps réel *push* aux opérateurs à l'arrivée d'une commande à leur poste.
- Module d'expédition complet, intégré aux API des transporteurs pour le suivi des colis.
- Gestion fine du stock de panneaux, en reliant les surfaces consommées à un stock de matière.
- Approfondissement du tableau de bord (prévision de charge, goulots d'étranglement).

11.3 Compétences acquises

Développement full-stack — PHP, JavaScript / Vue.js, SQL, API REST, expressions régulières.

Écosystème WordPress / WooCommerce — hooks, filtres, API REST, HPOS, rôles, plugin ACF.

Reprise de l'existant — lecture et portage de ~3 750 lignes de VBA vers PHP.

Modélisation métier — traduction de processus industriels en structures de données et machines à états.

Résolution de problèmes — débogage de bugs complexes (z-index, *falsy*, pollution ACF, imports silencieux).

Optimisation des performances — requêtes par lot, court-circuit, stratégies dégradées, files d'attente.

Méthodologie — développement itératif, tests terrain, versioning et déploiement maîtrisés.

Conclusion

Ce stage au sein du groupe Tikimob m'a permis de mener deux projets de développement complets et complémentaires. La **mission 1** a transformé un fichier Excel partagé en un véritable mini-ERP de production : le plugin « Suivi Commandes » centralise les commandes de plus de treize boutiques dans un tableau de bord unique, automatise les tâches manuelles les plus chronophages, offre à chaque opérateur une interface adaptée à son poste, et synchronise désormais les données dans les deux sens avec les sites d'origine. La **mission 2** a prolongé cette logique d'automatisation jusqu'aux fichiers d'usinage et de transport, en reprenant un outil Excel à macros pour l'intégrer au même plugin.

Le gain de productivité estimé — de l'ordre de 80 minutes par jour et par opérateur, soit près de 290 heures par an — illustre l'impact concret qu'un outil logiciel bien pensé peut avoir sur l'efficacité d'un atelier industriel. Au-delà du temps gagné, c'est la **qualité des décisions** qui s'améliore : un commercial dispose d'une vue consolidée, un opérateur usinage voit immédiatement les pièces à refaire, le bureau d'étude suit sans confusion les versions de plans.

Le projet continue de vivre : la généralisation des exports de la mission 2 sur tous les sites et l'enrichissement progressif du tableau de bord transformeront cet outil en une plateforme ERP à part entière pour l'ensemble du groupe. Le manuel de maintenance rédigé avant mon départ doit en garantir la continuité, quelle que soit la personne qui reprendra le projet.

Bilan professionnel

Développement du savoir-faire technique. Ce stage a considérablement enrichi mes compétences en développement logiciel. La maîtrise de l'écosystème WordPress / WooCommerce, la conception d'une API REST cohérente, l'import multi-sites avec stratégie dégradée et le portage d'un outil VBA vers PHP constituent autant de blocs réutilisables. La conception d'une machine à états et d'un système RBAC granulaire m'a confronté à la complexité des règles métier réelles, où chaque cas particulier (pièce à refaire, circuit carton, acompte/solde) doit être anticipé.

Renforcement du savoir-être. Au-delà du code, dialoguer avec les opérateurs pour comprendre leurs contraintes, traduire leurs besoins en spécifications, puis revenir vers eux pour valider mes choix a affiné ma capacité à vulgariser. Gérer un projet sous contraintes (déploiement progressif, bugs en production, attentes de la direction) a développé ma rigueur et ma capacité à prioriser.

Originalité et valeur ajoutée. Mon approche s'est distinguée par ma capacité à dépasser les exigences initiales : court-circuit `_fields=id`, stratégie d'import dégradée, anti-pollution ACF, ou encore l'intégration de la mission 2 directement dans le plugin plutôt que comme outil séparé. Cette proactivité témoigne d'une vision globale des enjeux organisationnels.

Bilan personnel

Apports personnels. Cette expérience a transformé ma vision du métier. Avant ce stage, je voyais le code comme une fin en soi. Travailler au plus près des opérateurs m’a appris que le code n’est qu’un **moyen** : ce qui compte, c’est l’outil livré et son adoption. Une fonctionnalité élégante mais peu utilisée vaut moins qu’un bouton trivial qui fait gagner cinq minutes par commande.

Projection. L’an prochain, je poursuis mes études **en alternance au sein du groupe Léa Nature**. Les missions y seront très différentes de celles de ce stage, davantage tournées vers l’exploitation et l’analyse de données que vers le développement d’un outil métier de A à Z. Je vois cette différence comme une chance : après avoir conçu et déployé une application complète en environnement industriel, je vais pouvoir confronter mes acquis à un autre contexte, d’autres outils et d’autres problématiques, et ainsi élargir mon profil.

Points d’amélioration. Ce stage a aussi révélé mes limites. Ma tendance au perfectionnisme technique m’a parfois conduit à passer trop de temps sur des optimisations marginales au détriment de fonctionnalités plus visibles. Par ailleurs, ma maîtrise des architectures distribuées demeure à approfondir : le plugin fonctionne bien à l’échelle actuelle, mais concevoir une solution pour un volume dix fois supérieur supposerait de maîtriser des concepts (files de messages, idempotence, partitionnement) que je dois encore consolider.

Au-delà du cadre de ce stage, cette expérience a directement nourri un **projet personnel** que je porte en parallèle : la création, à terme, d’une petite **agence technologique** destinée aux TPE/PME et e-commerçants. L’idée part exactement du même constat que celui rencontré chez Tikimob, mais généralisé : beaucoup de petites entreprises souffrent d’un « syndrome des données éclatées » — une boutique en ligne, un site vitrine, un logiciel de stocks et une base clients qui ne communiquent pas entre eux, d’où des ressaisies manuelles, des erreurs et du temps perdu. Mon offre consisterait à **automatiser ces tâches répétitives, connecter ces outils et centraliser les données** dans un point unique, avec à la clé des tableaux de bord de pilotage — exactement la démarche appliquée à l’échelle d’un atelier industriel.

Le positionnement que j’envisage repose sur trois idées simples. **(1) Une cible délaissée** : les grandes agences visent les grands comptes et les freelances se limitent souvent à des missions ponctuelles, ce qui laisse un espace pour accompagner durablement les petites structures. **(2) Une double compétence** no-code et développement sur mesure (API), qui permet de coder une vraie solution lorsque les outils clés en main atteignent leurs limites. **(3) Un modèle par abonnement** (maintenance et évolutions), gage de revenus récurrents et de fiabilité dans le temps. Le stage Tikimob constitue à mes yeux la **première brique concrète** de ce projet : il m’a prouvé, sur un cas réel, que la centralisation et l’automatisation de données créent une valeur immédiatement mesurable.

Glossaire technique

ACF

Champs personnalisés ajoutés aux commandes WordPress.

API REST

Interface permettant à deux applications de s'échanger des données via le web (HTTP).

Batch

Traitement par lot : plusieurs requêtes regroupées en une seule.

CRUD

Les quatre opérations de base sur des données : créer, lire, modifier, supprimer.

DSU

Date de Sortie d'Usine : date prévue de départ du produit fini.

EDI

Échange de Données Informatisé : fichier normalisé transmis aux transporteurs.

ENUM

Type de champ limité à une liste de valeurs prédéfinies.

Hook

Mécanisme WordPress exécutant du code en réaction à un événement.

HPOS

Mode de stockage performant des commandes WooCommerce.

Machine à états

Modèle décrivant les états d'un objet et les passages autorisés entre eux.

RBAC

Contrôle d'accès fondé sur les rôles des utilisateurs.

Regex

Expression régulière : motif servant à repérer du texte précis.

Repeater

Champ répétable stockant une liste (ici, les versions de plan).

SPA

Application web qui se met à jour sans recharger la page.

VBA

Langage de programmation des macros Excel.

Webhook

Notification automatique envoyée par un site lors d'un événement.

Extraits de code significatifs

B.1 — Stratégie d'import par lot dégradée (PHP)

```
1 $strategies = [  
2     ['fields' => 'id,status,date_created,total,total_tax,billing,  
3         shipping,customer_note,line_items,meta_data',  
4     'timeout' => 60, 'label' => 'complet'],  
5     ['fields' => 'id,status,date_created,total,total_tax,billing,  
6         shipping,customer_note,line_items',  
7     'timeout' => 45, 'label' => 'sans meta'],  
8     ['fields' => 'id,status,date_created,total,total_tax,billing,  
9         shipping,customer_note',  
10    'timeout' => 30, 'label' => 'minimal'],  
11 ];
```

B.2 — Court-circuit intelligent (sc-expose-meta.php)

```
1 function sc_analyze_requested_fields($request) {  
2     $fields = $request->get_param('_fields');  
3     if (empty($fields)) return ['only_id' => false, 'wants_meta' => true];  
4     $list = array_map('trim', explode(',', (string) $fields));  
5     if (count($list) === 1 && $list[0] === 'id')  
6         return ['only_id' => true, 'wants_meta' => false];  
7     return ['only_id' => false,  
8         'wants_meta' => in_array('meta_data', $list, true)];  
9 }
```

B.3 — Critère de regroupement des pièces (mission 2, VBA)

```
1 ' Petite piece si longueur < 350 OU largeur < 200  
2 If longueur < 350 Or largeur < 200 Then  
3     wsSrc.Rows(i).Copy Destination:=wsDest.Rows(destRow)  
4     destRow = destRow + 1  
5 End If
```